

PROGRAMACION DISTRIBUIDA

Sistemas distribuidos: Mapa conceptual

Héctor Pérez



2



RCSD: José M. Drake y Héctor Pérez

04/05/2015

Definición de Sistema Distribuido

*A collection of independent computers that appears to its users
as a single coherent system*

Distributed Systems: principles and paradigms, A. Tanenbaum & M. Van Steen

*A collection of autonomous computers linked by a computer network
with distributed system software*

Distributed Systems: Concepts and Design, G. Coulouris, J. Dollimore & T. Kindberg

*A system of multiple autonomous processing elements cooperating
in a common purpose or to achieve a common goal*

Real-time systems and programming languages, A. Burns & A. Wellings

*A distributed system is a system where I can't get my work done because
a computer has failed that I've never even heard of*

Leslie Lamport

¿Por qué distribuir el sistema informático?

- El sistema interactúa con un **entorno distribuido geográficamente**
- Hay **recursos** que son **compartidos** por muchas aplicaciones
- Se prevé **escalar** el sistema en varios ordenes de magnitud
- Se necesita **acumular** la **potencia** de **cálculo** de muchos computadores
- El sistema requiere **alta disponibilidad**

Objetivos

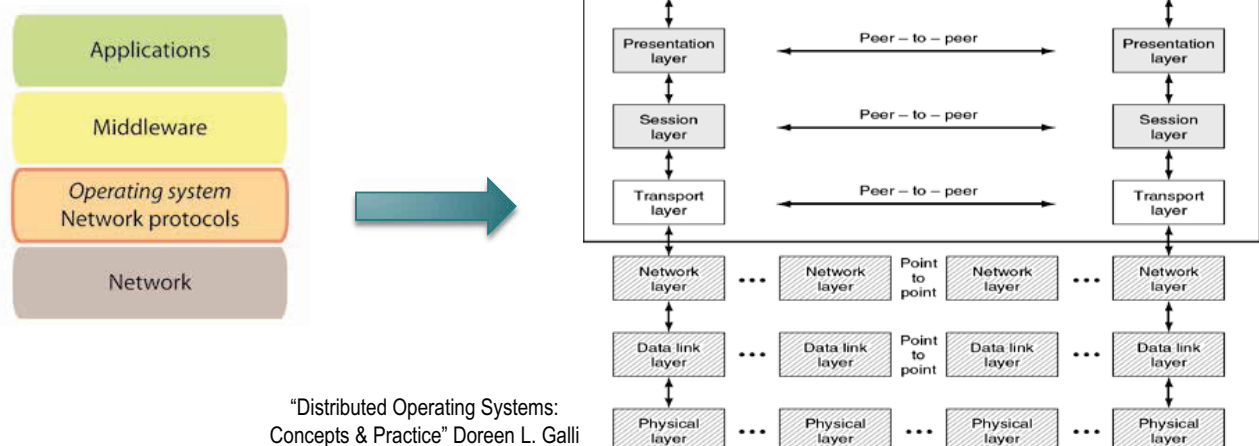
- Permitir el acceso a los recursos disponibles
 - *busca optimizar la eficiencia*
- Uso transparente de los recursos
 - *acceso local/remoto, concurrente, reubicación, múltiples copias*
- Tolerancia a fallos
 - *los fallos afectan parcialmente al sistema distribuido*
- Escalabilidad
 - *aumento de recursos/usuarios*
- Integración de sistemas heterogéneos

Arquitectura general (1/3)



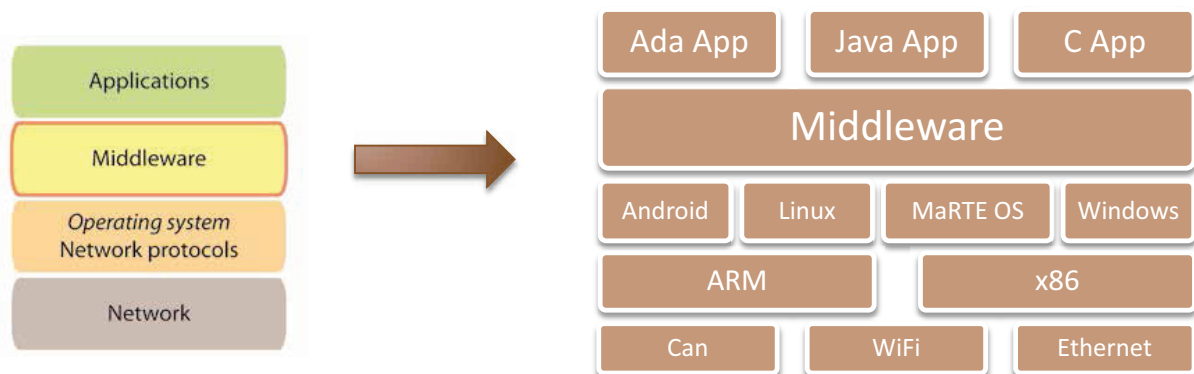
- La red constituye un **recurso compartido**
- Influye en las características del sistema distribuido: escalabilidad, rendimiento, movilidad, fiabilidad, calidad de servicio (QoS), etc.
 - Ej. Ethernet, WiFi, SpaceWire, Can, FlexRay

Arquitectura general (2/3)



- Un protocolo representa un conjunto de reglas para el intercambio de información
- Permite la utilización de diferentes redes
 - Ej. TCP/IP sobre Ethernet o sobre WiFi

Arquitectura general (3/3)



- Capa de software entre la aplicación y los servicios de red proporcionados por el sistema operativo
- Proporciona una abstracción de alto nivel de las comunicaciones
 - Gestiona el proceso de comunicación entre nodos
 - Permite la comunicación entre sistemas heterogéneos

Aspectos fundamentales en un sistema distribuido (1/3)

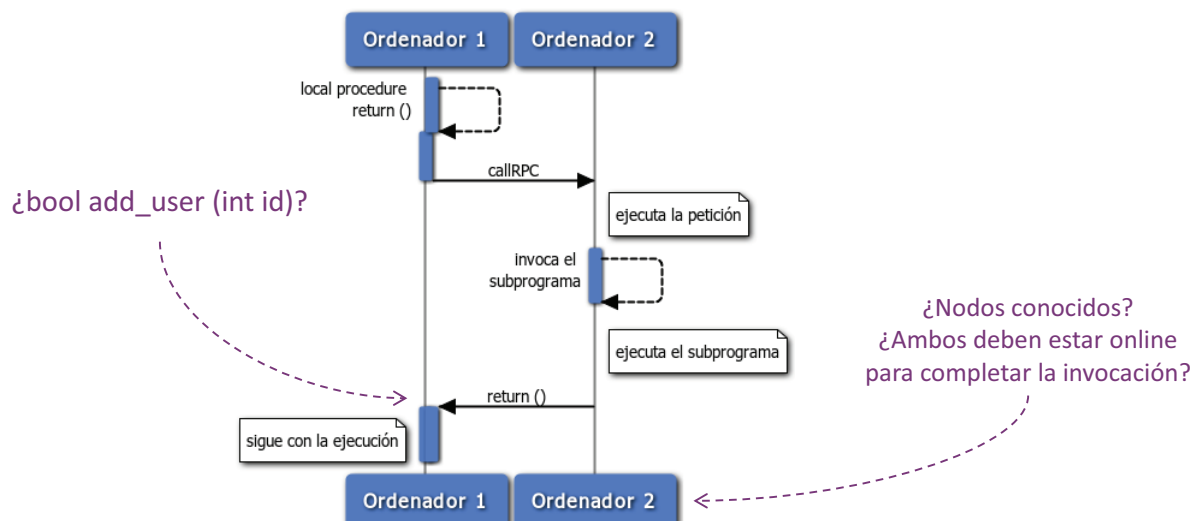
- ***¿Cómo se comunican las aplicaciones distribuidas?***
 - visión general de los paradigmas básicos de comunicación

Paradigmas de comunicación (1/3)

- Uso directo de los servicios de comunicación del SO
 - distribución explícita (ej. sockets)
 - propenso a errores en sistemas con múltiples nodos
 - complejidad inherente a un sistema real
 - ¿cómo gestionar la entrada/salida (I/O)?
 - ¿cómo gestionar componente dinámicos?
 - ¿cómo representar adecuadamente un mensaje por la red de comunicaciones?
 - ¿cómo gestionar los mensajes perdidos?
 - ¿necesito utilizar varios protocolos de comunicaciones? ¿o varios lenguajes de programación?

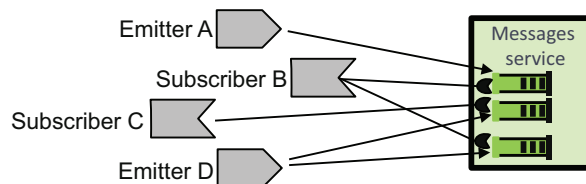
Paradigmas de comunicación (2/3)

- Comunicación basada en invocaciones remotas
 - ejemplo representativo: llamadas a procedimientos remotos (RPC)
 - invocación transparente de subprogramas



Paradigmas de comunicación (3/3)

- Comunicación indirecta
 - mayor desacoplo en las comunicaciones
 - **temporal**: no es necesario que el emisor y el receptor se ejecuten al mismo tiempo
 - **espacial**: no es necesario conocer la fuente o el destino de los mensajes
 - ejemplo: colas de mensajes

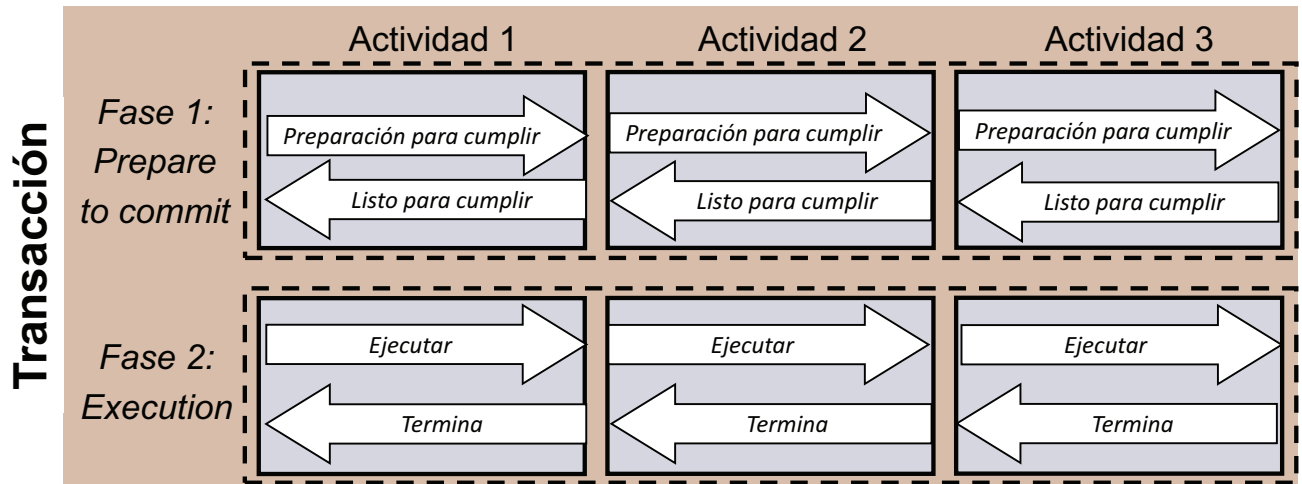


Aspectos fundamentales en un sistema distribuido (2/3)

- *¿Cómo se comunican las aplicaciones distribuidas?*
 - visión general de los paradigmas básicos de comunicación
- *¿Qué entidades componen un sistema distribuido?*
 - desde una perspectiva de diseño de alto nivel

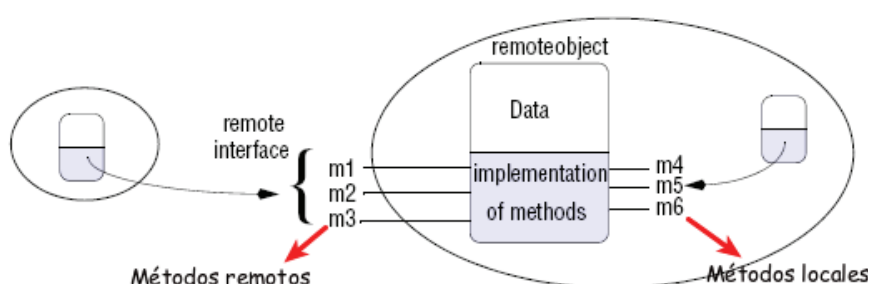
Abstracciones de alto nivel para la comunicación (1/3)

- Aplicaciones distribuidas basadas en transacciones
 - invocación coordinada del conjunto de actividades (ACID)



Abstracciones de alto nivel para la comunicación (2/3)

- Aplicaciones distribuidas basadas en objetos
 - uso transparente de objetos distribuidos
 - Objetos locales o remotos
 - Objetos remotos con métodos remotos y locales
 - Acceso remoto como si fuera local mediante la interfaz remota (*proxy*)



"Distributed Systems:
Concepts and Design"
George Coulouris et al.

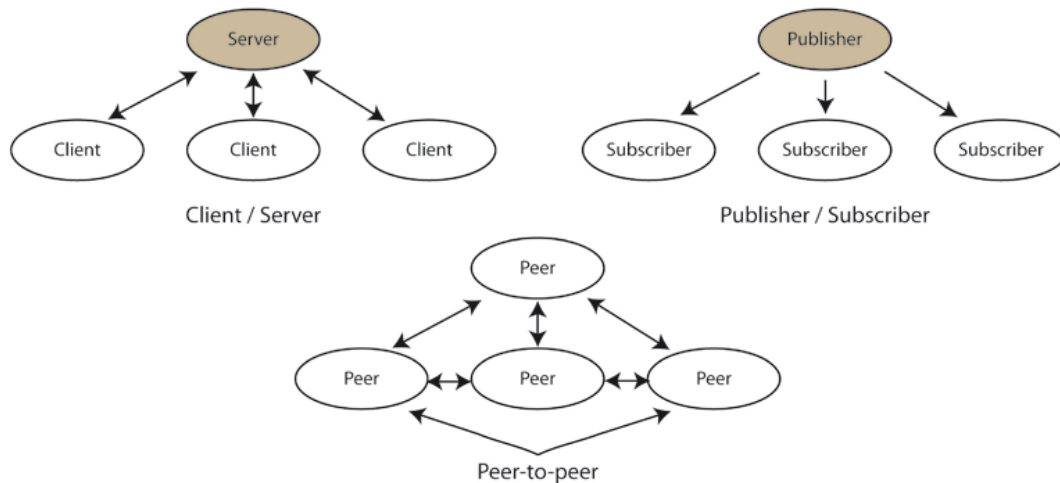
Abstracciones de alto nivel para la comunicación (3/3)

- Aplicaciones distribuidas basadas en componentes *reutilizables*
 - aislamiento
 - componibilidad
 - opacidad
- Aplicaciones distribuidas basadas en servicios
 - integración con la tecnología *www*
 - SOAP, XML, HTTP, REST, etc.
 - “business-to-business integration”

Aspectos fundamentales en un sistema distribuido (3/3)

- *¿Cómo se comunican las aplicaciones distribuidas?*
 - visión general de los **paradigmas básicos de comunicación**
- *¿Qué entidades componen un sistema distribuido?*
 - desde una perspectiva de diseño de alto nivel
- *¿Qué función desempeña cada nodo?*
 - visión general de los **paradigmas básicos de interacción**

Paradigmas de interacción



¿Qué función desempeña cada nodo del sistema distribuido?

Arquitectura cliente/servidor

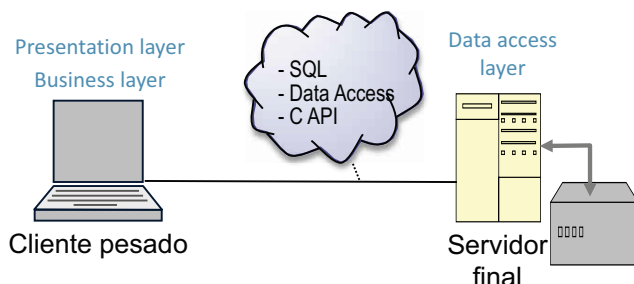
- La *arquitectura cliente/servidor* es un paradigma de interacción entre los elementos que constituyen una aplicación distribuida:
 - Clientes:** elementos *activos* que dirigen las actividades que deben ejecutarse para implementar la tarea requerida por la aplicación. Realizan peticiones a los servidores para que ejecuten algunas de esas actividades
 - Servidores:** elementos *pasivos* especializados en realizar ciertas tareas bajo petición de los clientes. Habitualmente representan elementos que son compartidos por múltiples clientes, de una o varias aplicaciones
- Proporciona un *marco de referencia* sencillo, flexible y abierto para distribuir la ejecución de una aplicación en varios nodos de una plataforma. Se caracteriza por el acoplamiento entre elementos

Arquitectura cliente/servidor: Características

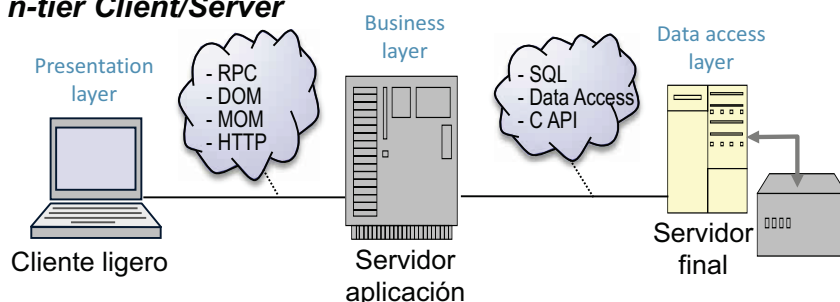
- **Servicios:** Facilita la colaboración de procesos que se ejecutan en diferentes máquinas, a través de intercambios de servicios. Los procesos servidores proveen los servicios, los clientes los consumen
- **Recursos compartidos:** Los servidores pueden ser invocados concurrentemente por los clientes y, por tanto, debe arbitrarse el acceso a sus recursos compartidos
- **Comunicación asimétrica:** Un servidor puede atender a múltiples clientes. El cliente conoce el servidor que invoca. El servidor no necesita conocer el cliente que atiende
- **Independencia de la ubicación:** La ubicación de los servidores es transparente al cliente. Se utilizan servicios de localización definidos a nivel de plataforma para que los clientes encuentren a los de servidores
- **Soporte de clientes y servidores heterogéneos:** Los mecanismos de interacción entre clientes y servidores son independientes de las plataformas. El middleware independiza la aplicación de la plataforma

Cliente/servidor: Estrategias de reparto de la complejidad

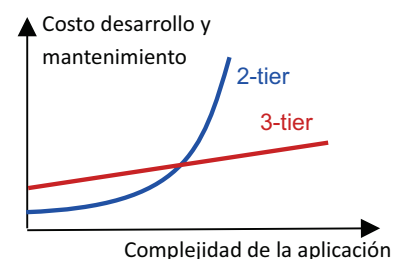
2-tier Client/Server



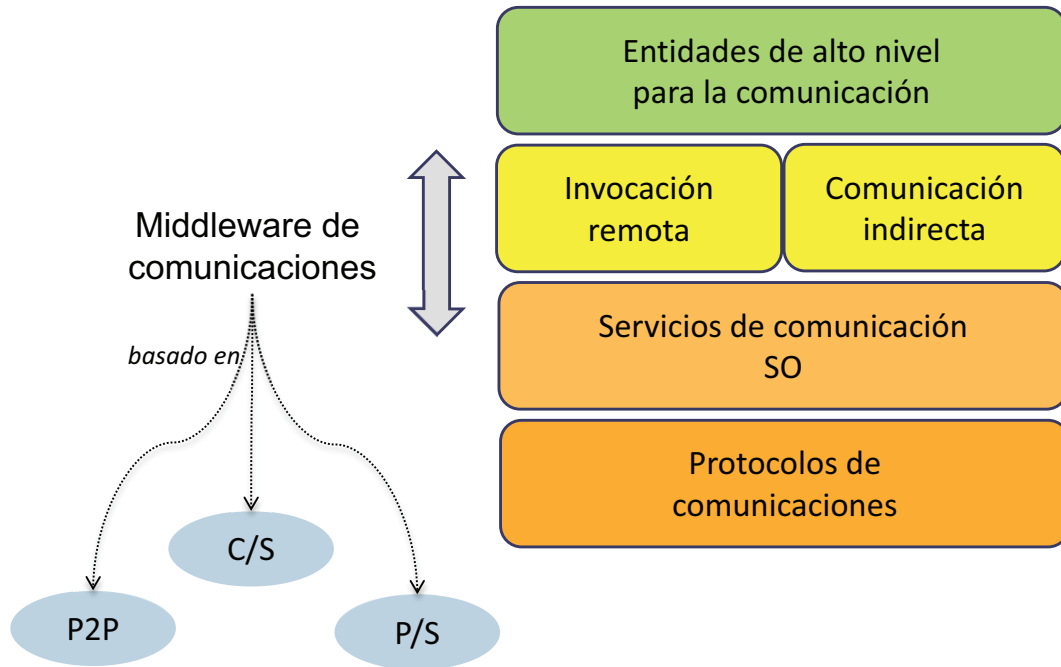
n-tier Client/Server



2-tier versus n-tier



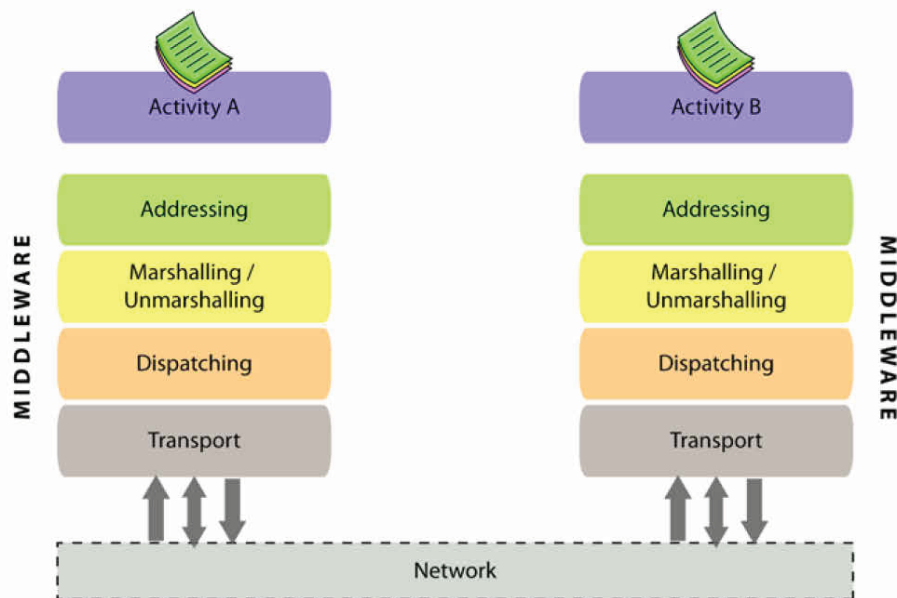
Middleware de comunicaciones



Middleware de comunicaciones: Características

- Gestiona todos los detalles de bajo nivel relacionados con las comunicaciones (ej. sockets)
 - permite la comunicación transparente entre aplicaciones a nivel de usuario
- Gestiona las diferencias relacionadas con el hardware, los sistemas operativos y los protocolos de comunicaciones
 - facilita la comunicación entre sistemas heterogéneos
- Habitualmente proporciona una interfaz estandarizada para el desarrollo de aplicaciones
 - facilita el desarrollo y la interoperabilidad entre aplicaciones
- Proporciona un conjunto de servicios habituales en sistemas distribuidos
 - servicios de localización de entidades (naming), registro de logs, etc.

Middleware de comunicaciones: Estructura (1/2)



Middleware de comunicaciones: Estructura (2/2)

- **Addressing** o la asignación de referencias que indiquen la ubicación de cada entidad distribuida
- **Marshalling** o la transformación de datos a un formato adecuado para su transmisión por la red de comunicaciones
- **Dispatching** o la asignación de recursos para el procesamiento de cada invocación
- **Transport** o el establecimiento de un enlace de comunicaciones para el intercambio de mensajes

Tecnologías middleware

